~~df.iloc [[0,10],1:7~~
df.iloc [0:10, 1:7]

# Extracting top 1 to top 10 ~~from~~ column age : weight.
columns = [1,2,3,4] #i.e Age dob height weight
# getting top 1 to 10 with column from Age to Weight.
df.iloc [0:10, columns]

5. Selecting with condition.
NB iloc cannot accept [a boolean series], but only a boolean list.
- Therefore we have to use the [list()] function to convert a series into a boolean list.

a) One condition
= Select ~~a~~ player with height above 180cm.
columns = [1,2,3,4]
df[height] > 180 # condition but in boolean series.
list (df["height"] > 180) #converted to boolean list.
df.iloc [list(df["height"] > 180), columns]

b) Multiple condition columns = [1,2,3,4]
= Player with height above 180cm and argentinian.
((df["height-cm"] > 180) & (df["nationality"] == "Argentina"))
df.iloc [list((df["height-cm"] > 180) & (df["nationality"] == "Argentina"))
, columns]

#Set new Value for a cell in dataframe.

```
import pandas as pd.
df = pd.read_csv(r"players_20.csv")
#set index
df.set_index("short_name", inplace=True)
#selecting columns.
df= df[["long_name", "age", "dob", "height_cm", "weight_kg",
        "nationality", "club"]]
df.head()
```

1. #set value to one cell

```
#update L.Messi height.
df.loc["L.Messi", "height_cm"] #locate the cell (170)
df.loc["L.Messi", "height_cm"]=175
#show dataframe.
df
```

2. #set value to an entire column.

```
#height of all players to 190cm.
df.loc[:, "height_cm"]=190  #(:) get all rows.
df.
```

3. #set value to an entire row.

```
#get all columns that correspond to player ranked last in
fifa.
 df.iloc[-1,:]  #All the data about the last player.
#set null values to the columns of the last player.
import numpy as np.
np.nan
df.iloc[-1,:]=np.nan.
df
```

4# Set value to multiple cells.
#set value for all items matching the list of labels.
df.loc[["L.Messi", "Cristiano Ronaldo"], ["height_cm"]] = 178
df

5# Set value for rows matching a ~~column~~ condition
#set value for rows matching a condition.
columns = ['age', 'dob', 'height-cm', 'weight-kg'].
#Condition.
df["height-cm"] > 180
#extracting.
df.loc[df["height-cm"] > 180, columns]
#set value.
df.loc[df["height-cm"] > 180, columns] = 0
df

#Drop Rows or Columns from a DataFrame.
import pandas as pd.
df = pd.read_csv(r"players_20.csv")
df.set_index("short_name", inplace = True)
df[["long_name", "age", "dob", "height_cm", "weight_kg", "nationality", "club"]]
#Drop Row.
1. a.) Drop one row using axis parameter (axis=1 ⇒ column) (axis=0 ⇒ Row)
df.drop("L.Messi", axis=0)
b.) Drop one row using index parameter.
df.drop(index = ["L.Messi"])

2. a.) Drop two or more rows & update data (inplace=True)
df.drop(["L.Messi", "Cristiano Ronaldo"], axis=0, inplace=True)
df

Drop columns:

1 a) Drop one column with axis parameter.

```
df.drop("long_name", axis=1)
```

b) Drop one columns with column parameter.

```
df.drop(columns=["long_name"])
```

c) drop column by position/index (drop last column)

```
df.columns[[-1]]  # locating the column.
df.drop(df.columns[[-1]], axis=1)
```

2 a) Drop two or more columns and update data.

```
df.drop(["long_name", "dob"], axis=1, inplace=True)
df
```

How to Create random sample using sample method. (sample())

```
df = pd.read_csv("players_20.csv")
# extract 10 random elements from "nationality" column.
df["nationality"].sample(10)
```

NB its random.

We can control the random number generation in sample method by writing operator named (random_state) = any number we want. parameter

```
df["nationality"].sample(10, random_state=99)  # Remain the same.
```

Extract a random 20% sample of the dataframe.
check no. of rows.

```
df.sample(18278/5)
```

or we can use the following (frac parameter)

```
df.sample(frac=0.2, random_state=12)  # remains same.
```

increasing sampling rate (upsampling)

= We have to specify the frac parameter i.e (frac > 1)

NB When we want to upsample for (frac parameter > 1) the replace parameter in the sample method has to be set to **True**

Double upsampling.

```
df.sample(frac = 2, replace = True)
                    random-state = 99
```

= The dataframe should double the number of rows. of the original dataframe.

Filtering a DataFrame with the query method.

```
import pandas as pd.
df = pd.read_csv(r"players-20.csv")
#set index.
  df.set_index("short-name", inplace = True)
#select column.
  df = df[["long-name", "age", "..." ...]]

#Using query method.(Players older than 28)
  df.query("age>28")
#With boolean slicing will be. == query method.
  df[df["age"]>28]

#Query method for more than one constraint.
 #Select players older than 32 and from Spain.
  df.query("age>32 and nationality == 'Spain'")
```

NB for boolean slicing we use (&) while in query method we use (and)

```
#Boolean slicing
df[(df["age"]>32)&(df["nationality"]=="Spain")]
```

# Query age not greater than 34
df.query("not(age>34)")
# for boolean(not) == (~)
# Convert height into m and select those above 1.8.
df.query("height_cm/100>1.8")

df.query("height_cm/100>1.8")

Select players that were born after 1990.
1. Check out datatype of that column we are to query.
df["dob"].dtypes / df.dtypes
2. Convert "dob" column to datetime type.
df["dob"]=df["dob"].astype("datetime64") # df["dob"].dt.year
3. Query.
df.query("dob.dt.year>1990")

⇒ df.dtypes ⇒
⇒ df["dob"] = df["dob"].astype("datetime64")
⇒ df.query("dob.dt.year>1990")

You have to know that columns attribute.
Apply function / method in pandas. (players_20.csv)

= Helps us apply functions/operations to a column/dataframe
= Both built-in function and our-functions.
# Built-in function to a column/series.
import numpy as np.
df["age"].apply(np.sqrt)

Create our own function and applying it to a df.

```
def calculate_bmi(row):                    # row represents the dataframe
    return row["weight_kg"] / (row["height_cm"]/100)**2

df.apply(calculate_bmi, axis=1)
```

Lambda function + apply() method. (players_20.csv)

```
# Create a sum function
def sum_values(a,b):
    x = a+b
    return x
```

Calling function

```
sum_values(4,3)
= 7
```

```
# Create a lambda function (one-liner)
sum_values_lambda = lambda a,b : a+b
sum_values_lambda(4,3)    = 7
```

NB # colon in lambda seperates input and output.

NB # It is useful to use lambda function if you want to create a temporary function.

= Also useful for simplification since its a one-liner.

```
# Use lambda function to convert height into metres.
df["height_cm"].apply(lambda x : x/100)
```

NB x represents every row in the series "height_cm"

```
# Use lambda function to convert "long_name" to upper case.
df["long_name"].apply(lambda x : x.upper())
```

NB # But make sure its a string(x) datatype since we use upper method

```
# Alternative. (specify an attribute hence hard)
df["long_name"].str.upper()
```

Use lambda function to get the year on "dob" series.
df["dob"].apply

Convert the datatype of the column to datetime64.
df["dob"] = df["dob"].astype("datetime64")

Use lambda function:
df["dob"].apply(lambda x: x.year)

Alternative (we used .dt attribute)
df["dob"].dt.year

# Apply lambda function to df to calculate bmi
df.apply(lambda x: x["weight-kg"] / ((x["height-cm"]/100)**2), axis=1)

# Copy method. (players-20.csv)
# By default deep=True Deep copy
df_copy = df.copy(deep=True)

NB When deep parameter == True, All the modification we make to the original dataframe is not going to be reflected in the copy and viceversa

Update C. Ronaldo's height.
df.loc["Cristiano Ronaldo", "height-cm"] = 198
Check both dataframes.
df # 198
df_copy # Remains 190

Shallow copy

NB When we set the parameter deep=False all the modification we make to the original dataframe is going to be reflected in the copy dataframe and viceversa.

df_copy_false = df.copy(deep=False)
df.loc["L. Messi", "weight-kg"] = 68

#check both df.

```
df                          # 68 kg
df.copy=false               # 68 kg.
```

Type 3 of copy == Simple Assignment.

3 = Simple Assignment copy is also a shallow copy.

```
df_new_copy = df
df.loc["L. Messi", "height_cm"] = 180
df                          # 180   Hence a shallow copy.
df_new_copy                 # 180
```

Pivot Tables

There are two methods to create pivot tables

1.) Pivot() → Reshapes data based on column values (does not support data aggregation)

Data Aggregation → taking raw data and summarizing it so its easier to understand & analyze.

Example (df)

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t

```
df.pivot(index="foo",
         columns="bar",
         values="baz")
```

(Result)

bar	A	B	C
foo			
one	1	2	3
two	4	5	6

2) Pivot_table() → Creates a spreadsheet-style pivot table (supports data aggregation)

1. # Pivot() method.

```
import pandas as pd
df_gdp = pd.read_csv("gdp.csv")

# reshaping using pivot method.
df_gdp.pivot( index = "year",
              columns = "country",
              values = "gdppc" )
```

2. # Pivot_table() method.

```
import pandas as pd.
df_sales = pd.read_excel("supermarket_sales.xlsx")
# Goal is to see how much male & female spend their
  money in this supermarket.
# Make a pivot table and add an aggregate function.
df_sales.pivot_table(index = "Gender", aggfunc = "sum")
```

All columns

NB Aggfunc() is only for numerical data

```
# make a pivot table, aggfunc and select columns we only need.
df_sales.pivot_table( index = "Gender", aggfunc = "sum"
                      values = ["Quantity", "Total"] )
```

Shift + Tab

```
# create a pivot table to see in which category is
  money spent & how much.
df_sales.pivot_table( index = "Gender",
                      columns = "Product line",
                      values = "Total",
                      aggfunc = "sum" )
```

Project #2: Making Static and Interactive Data Visualization with Pandas.

- Make a dataframe / Create a pivot table } so as to easily make the plots with pandas.

```
import pandas as pd
df_pop_raw = pd.read_csv(r"population_total.csv")
    df_pop_raw
# Drop all null values. (dropna() method)
    df_pop_raw.dropna(Inplace=True)
# Making a pivot table.
df_pivot = df_pop_raw.pivot( index="year", columns="country",
                             values="population")

# selecting some countries.
df_pivot = df_pivot[["United States","India","China","Indonesia","Brazil"]]
    df_pivot
```

Visualization. (Static)

1. Lineplot

```
df_pivot.plot(kind="line", xlabel="YEAR", ylabel="Pop",
              title="Population (1955-2020)")
```

more parameter `figsize=(8,4)` (8: x axis, 4: y axis)

2. Barplot

Single Barplot (one year)

```
df_pivot_2020 = df_pivot[df_pivot.index.isin([2020])]

df_pivot_2020.
```

NB For making barplots, we have to put text data in the index hence we transpose the dataframe

Switch

Transpase → changing rows to columns or columns to rows.

= Its an attribute

```
df-pivot-2020.T
df-pivot-2020 = df-pivot-2020.T
# Making the Barplot
df-pivot-2020.plot(kind="bar", color="orange",
                   xlabel="Year", Ylabel="Pop"
                   title="Population 2020");
```

```
# Barplots grouped by "n" variables;
# Not just one year.
df-pivot-sample = df-pivot[df-pivot.index.isin([1980, 1990,
                  2000, 2010, 2020])]
Transpose.
df-pivot-sample.plot(kind="bar", xlabel="Year", Ylabel="Pop");
```

2# Piechart

```
df-pivot-2020   # it is already transposed into bargraph type.
```

NB Since 2020 is the column name which is integer, we have to make it into a string

```
df-pivot-2020.rename(columns={2020:"2020"},
                     inplace=True)
#Making the plot
df-pivot-2020.plot(kind="pie", y="2020")
```

3# Boxplot.

- Shows how data is spreadout.
- Show 5 important numbers 1. Minimum 3. Median 5. Maximum value.
  2. First Quantile 4. Third quantile

Single boxplot.
df-pivot ["United states"].plot(kind="box", color="green", ylabel="Pop")

Multiple boxplot.
df-pivot.plot(kind="box", ylabel="Population")

4. # Histogram. (Data points → Ranges)
df-pivot ["Indonesia"].plot(kind="hist")

df-pivot [["Indonesia", "United states"]].plot(kind="hist")

5. # Scatterplot
=Plot datapoints on a horizontal/vertical axis.
df-pop-raw [
X= df-pop-raw["Country"].isin(["United states", "India", "China", "Indonesia"])

df-sample = df-pop-raw[x]

df-sample.plot(kind="scatter", x="year", y="pop")

How to export pivot table and plots we make with pandas.
1. Exporting plots we make with pandas
import matplotlib.pyplot as plt
#Save the plot
plt.savefig("my-test.png") #"my-test.png"
mytest= name of the file
png= extension.
# show plot
plt.show() # to remove words on low side of the graph.
It also exports the figure.

NB= The file should be located in the same folder where we have our jupyter notebook file

2. Exporting the pivot table.

```
df-pivot.to-excel("pivot-table.xlsx")
```

Interactive Visualization with Pandas.

- We need to install cufflinks library which connect Pandas and plotly so as we can make i. visualiza in Pandas
- No need to learn plotly syntax

```
# installing plotly.
!pip install plotly  #(!) function is → to run command in Jupyter Notebook
# installing cufflinks.
!pip install cufflinks

import pandas as pd
import cufflinks as cf
from IPython.display import display, HTML
# display and html helps us display the i. visualization in jupyter notebook.

#Editing default configuration of the cufflinks.
cf.set_config_file(sharing="public", theme="ggplot",
                   offline=True)
#To choose "cufflinks" themes we use getThemes() method
cf.getThemes()

#To use interactive visualization we use iplot() instead of plot()
df.iplot()
```

shift + Tab

```
df_pop_raw = pd.read_csv("population_total.csv")
# Dropping null values.
df_pop_raw.dropna(inplace=True)
# Making a pivot table.
df_pivot = df_pop_raw.pivot(index="year",
                            columns="Country",
                            values="population")
# Selecting some countries.
df_pivot = df_pivot[["United States", "India", "China", "Indonesia",
                     "Brazil"]]
df_pivot

# 1. Line Interactive plot.
df_pivot.iplot(kind="line", xTitle="Years", yTitle="Pop")
```

#NB for static visualization the horizontal labelling was (xlabel) while in interactive visualization its (xTitle)

= Interactive plotting comes with
- ⇒ Ability to download the plot.
- ⇒ Ability to zoom in and out
- ⇒ Ability to autoscale - Returning to its usual plot
- ⇒ Reset axis ⇒ fixes the axis position
- ⇒ Pan → helps us move in the plot
- ⇒ Has a link to plotly website.

```
# 2. Interactive Barplot. (Single)
df_pivot_2020 = df_pivot[df_pivot.index.isin([2020])]
#Transposing
df_pivot_2020 = df_pivot_2020.T
df_pivot_2020.iplot(kind="bar", color="lightgreen",
                    xTitle="Years", yTitle="pop")
```

#2.2 Barplots with more than one variable (year)

```
df_sample = df_pivot[df_pivot.index.isin([1980, 1990, 2000,
                                          2010, 2020])]

df_sample.iplot(kind="bar", xTitle="Years", yTitle="Pop")
```

#3. Piechart. interactive.

```
df_pivot_2020.rename(columns = {2020: "2020"},
                     inplace=True)
df_pivot_2020 = df_pivot_2020.reset_index()
df_pivot_2020.iplot(kind="pie", values="2020", labels="country")
```

NB: for interactive piechart the country series which was set as index have to be changed into columns and the df should have numbered index column.
= You can remove a country in the piechart by clicking it in the keys.

#4. Interactive boxplot

```
# Single boxplot
df_pivot["United States"].iplot(kind="box")

# Multiple boxplot
df_pivot.iplot(kind="box", yTitle="Pop")
```

#5. Interactive Histogram

5.1 One country.

```
df_pivot["United States"].iplot(kind="hist", xTitle="Population",
                                bins=3)
```

bins = number of bars in a graph.

5.2 Multiple Histograms.

```
df-pivot [["United States", "Indonesia"]].iplot(kind=
```

6. Interactive scatterplot.

```
df-pivot.iplot(kind="scatter", mode="markers", xTitle=
                "Years", yTitle="Pop")
```

GroupBy and Aggregation of a dataframe.

```
import pandas as pd.
df-cars = pd.read_csv(r"car-sales.csv")
df-cars = df-cars[["Manufacturer", "Sale_in_thousands",
          "Vehicle_type", "Price_in_thousands",
          "Engine_size", "Horsepower", "Fuel_capacity"]]
df-cars
df-cars.nunique()
```

1. Agg() method.

```
# Calculate sum of the columns
df-cars.agg("sum")
# Calculate mean of numerical columns
df-cars.agg("mean")
# Calculate mean and count values.
df-cars.apply(["mean", "count"])
# Different aggregation per column (sum & mean of "Sale in thousands" and max of "Price-in-thousands")
df-cars["Sale-in-thousands"].apply(["sum", "mean"])
```

or

```
df-cars.agg({"Sale-in-thousands": ["sum", "mean"],
             "Price-in-thousands": ["sum", "max"]})
```

Aggregate over the columns.

df_cars[["Sales_in_thousands", "Price_in_thousands"]].agg("sum", axis=1)

or

df_cars["Sales_in_thousands"] + df_cars["Price_in_thousands"]

These are actually same but for agg() method it takes null values (NaN) as 0 compared to the second one where if you sum the two columns one having null values the result will be null (Nan).

Aggregate different functions and rename the index of the resulting dataframe. (# Using tuples)

df_cars.agg(x=("Sales_in_thousands", "sum"),
y=("Price_in_thousands", "sum"))

or similar but the first has x and y as indexes

df_cars.agg({"Sales_in_thousands", ["sum"],
"Price_in_thousands", ["sum"]})

while second one has sum as index

The split-Apply-Combine strategy.

- This strategy helps us group values in a dataframe so we build categories and we can apply some aggregate function, so we see the values of those categories.

split data into seperate groups.

df_cars["vehicle_type"].value_counts() Passenger Car:

passenger_filter = df_cars["vehicle_type"] == "Passenger"

car_filter = df_cars["vehicle_type"] == "Car"

Apply an operation / function:

df_cars[passenger_filter] # gives us a dataframe with vehicle type passenger only.

#lets say you want to find mean of "Sales in thousands" of passenger vehicles. (Applying a function)

passenger_mean = df_cars[passenger_filter]["Sales_in_thousands"].mean()
car_mean = df_cars[car_filter]["sales_in_thousands"].mean()

combine result.

pd.DataFrame({"Vehicle_type": ["Car", "Passenger"],
"Sales_in_thousands": ["Passenger_mean",
"Sales_in_thousands": ["car_mean", "Passenger_mean

pd.DataFrame({"Vehicle_type": ["Car", "Passenger"],
"Sales in thousands": [car_mean, Passenger mean]})

This is easier by using GroupBy method. (1-liner)

groupby() method.

- it allows us group data into categories and apply functions inside each category.

# group by vehicle_type and calculate the mean
df_cars.groupby("Vehicle_type").mean()
# groupby is a lot easier than split-apply-combine strate

# groupby manufacture
groupby_obj = df_cars.groupby("Manufacturer")
groupby_obj.groups.keys()
# get a specific group within the manufacturer column.
groupby_obj.get_group("Ford")

```
#groupby manufacturer and calculate the mean.
df-cars.groupby("Manufacturer").mean()
#group by vehicle type and count-values.
df-cars.groupby("Vehicle-type").count()
#find null values in columns.
df-cars.isnull().sum()
```

NB If we want to consider null values i.e not to drop them then inside the groupby() method, we set the dropna parameter = False and then count how many null values we have

```
#groupby Engine-size and count values without dropping NaN values.
df-cars.groupby("Engine-size", dropna=False).count()
```

Using both groupby & agg() method to perform more complex operations on a dataframe.

```
#find minimum and maximum values on each column.
#grouping vehicle type.
df-cars.groupby("Vehicle-type").min()
df-cars.groupby("Vehicle-type").max()
#How to work out both of them at the same time using
  agg method.
df-cars.groupby("Vehicle-type").agg(["min","max"])
#find the minimum Engine-size & max Horsepower.
#set a name for aggregated columns (tuples)
df-cars.groupby("Vehicle-type").agg(min-eng=("Engine-size","min"),
                                    max-horse=("Horsepower","max"))
```

Calculate the sum of sales-in thousands and mean of price-in-thousands. (groupby manufacturers)

```
df_cars.groupby("Manufacturer").agg(sales=
        sum_sales=("sales_in_thousands", "sum"),
        mean_price=("Price_in_thousands", "mean"))
```

Alternatively we can use:

```
df_cars.groupby("Manufacturer").sum()["sales_in_thousands"]

df_cars.groupby("Manufacturer").mean()["Price_in_thousands"]
```

Previously we applied built-in functions to our dataframe e.g agg(), mean(), sum() now we want to be able to apply our own function to the dataframe using Lambda (one-liner)

Lambda.

e.g `df_cars.groupby("Manufacturer").sum()`

Prices_in_thousands & sales_in_thousands converted to its original value (eg 79.005 == 79.005x1000)

```
ars = df_cars.groupby("Manufacturer").sum().apply(lambda x: x*1000)
-price
df_cars.rename(columns={"sales_in_thousands": "sales",
                        "Price_in_thousands": "price"},
               inplace=True)

df_cars_sales_price[["sales_in_thousands", "Price_in_thousands"]]
```

Substracts values from the mean of each group
df-cars.groupby("Manufacturer").apply(lambda x: x-x.mean())
Alternatively
df-cars.groupby("Manufacturer").mean()

(df_cars) -(df-cars.groupby("Manufacturer").mean())

Filtering values based on an aggregate function.
filter method()
Calculate the mean of Sales-in-thousands.
df_cars["Sales-in-thousands"].mean()
52
# filter certain values out based on sum() and return a subset.
def filter-func(x):
return x["Sales-in-thousands"].sum() > 52
We filtered out companies that have their sum less than the mean of Sales-in thousands <= 52
df_filter=
df-cars.groupby(["Manufadurer"]).filter(filter-func)

df-filter ⇒ Dataframe with manufacturer who the sum of sales > 52.

# Finding values that are filtered out.
def filter_not(x):
return x["Sales-in-thousands"].sum() < 52

df_out = df_cars.groupby(["Manufacturer"]).filter(filter_not)

Alternatively.

df_cars[df_cars.groupby(["Manufacturer"]).sum()["Sales_in_thousands"].sort_values().head(6)]

then

You can check Manufacturer whose sum is less than 52 then use head() method and then filter out e.g 6 companies = head(6)

Merging and Concatenating Dataframes.

1. Concatenating Vertically.

import pandas as pd.

df_movies = pd.read_csv("IMDb movies.csv", low_memory=False)

df_ratings = pd.read_csv("IMDb ratings.csv")

df_movies = df_movies[["imdb_title_id", "title", "year", "genre", "country"]]

df_ratings = df_ratings[["imdb_title_id", "total_votes", "mean_vote"]]

Extract 50% sample of the original dataframe (df_movies)

sample = df_movies.sample(frac=0.5)

#Shape of df_sample

df_sample.shape

df_movies.shape

Concatenate the (df)s

df_concat = pd.concat([df_movies, df_sample], axis=0)

df_concat.shape

Vertically concatenation requires the dataframes to have similar columns.

2. Concatinating horizontally.

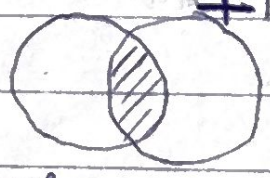
NB Different columns, same indexes hence can be concaterated horizontally.

= Axis = 1 } Horizontally.

```
# shape of the dataframe
Print(df_movies.shape)      #same number of rows/index
Print(df_ratings.shape)     But different no. of columns.

#concatenate the (df)s on "imdb_title_id" (horizontally)
        pd.concat([df_movies, df_ratings])

#But first they have one same column (imdb_title_id) hence
 we have to set that column as an index.
df_movies.set_index("imdb_title_id", inplace=True)
df_ratings.set_index("imdb_title_id", inplace=True)

df_concat_hor = pd.concat([df_movies, df_ratings], axis=1)

df_concat_hor.shape
```

Joins.

1# Inner join.



- There must be a common column between two data sets for there to be an inner join

Formula = df1.merge(df2, on="id", how="inner")

- Output = Dataframe that has data which is both in df1 and df2.

merge df_movies & df_ratings (inner join)
= They have the same one column (imdb_title_id)

~~df_movies.m~~
df_inner = df_movies.merge(df_ratings, on="imdb_title_id",
how="inner")

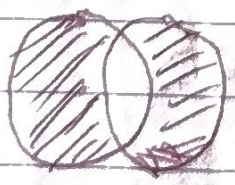
# Alternatively:
pd.merge(df_movies, df_ratings, on="imdb_title_id")

Full joins / Full outer join
= Allows us to join all the elements between two tables or dataframes

= There must be a common column.
NB = Full joins comes with a df having null values.
Formula = df1.merge(df2, on="id", how="outer")
column common

df_outerjoin = df_movies.merge(df_ratings, on="imdb_title_id",
how="outer")

3# Exclusive full joins.
= Exclusive full join will only pick the values that are exclusive to the table.



= There must be a common column

Formula df1.merge(df2, on="id", how="outer",
indicator=True).query( )
common column

= (Indicator=True) parameter helps us know which values are exclusive to df1 and which values are exclusive to df2.

- After exclusive full joining dataframes, we are going to get a new column in our output dataframe and its gonna be named -merge by default.

= under the column (-merge) the values will be
  = left_only => data exclusive to df1
  = both => elements both in df1 and df2.
  = right_only => elements that are exclusive to df2.

- After this, we use the query method to get values that are left_only and right_only.

```
df1.merge(df2, on="id", how="outer", indicator=True).
    query("_merge=='left_only' or _merge=='right_only' ")
```

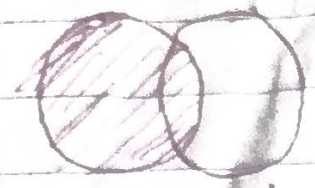
Exercise
Merge df_movies and df_rating with exclusive full join

```
df_movies.merge(df_ratings, on="imdb_title_id", how="outer",
    indicator=True).query
    ("_merge=='left_only', _merge=='right_only' ")
```

NB if both the dataframes have (one similiar column whereby that column from df1 has its values all similiar to that of df2 and we perform exclusive full join on it, the result after querying will be an empty data frame since the (_merge) column values is all (both))

#4. Leftjoin.

= Allows us to grab only the data that belongs to one table



= There must be a common column

formula = df1.merge(df2, on="id", how="left")

Exercise.

#Extract a 50% sample of the df-movies dataframe.

df_mov_sample = df_movies.sample(frac=0.5)

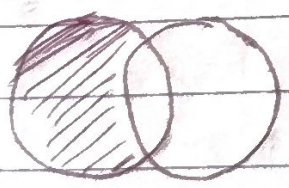
#Merge df_mov_sample and df_ratings (left join)

df_left_join = df_mov_sample.merge(df_ratings, on="imdb_title_id", how="left")

Print(df_left_join.shape)

#5. Exclusive left join.

= Allows us to grab data that is exclusive to one df.



= Must have a common column.

Formula df1.merge(df2, on="id", how="outer", indicator=True).query()

Exercise

#Make a copy of the df_movies

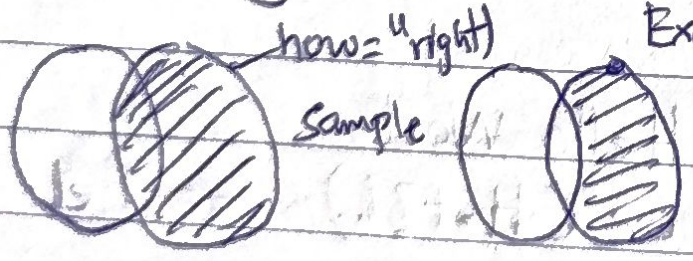
df_copy = df_movies.copy(deep=True)

#Set the first 1000 values of "imdb_title_id" as "tt1234567890"

df_copy.

```
for index in df_copy.index:
    if index < 1000:
        df_copy.loc[index, "imdb_title_id"] = "tt1234567890"

df_copy.

# Merge df_copy and df_ratings (exclusive left join)
df_ex_left = df_copy.merge(df_ratings, on="imdb_title_id", how="outer",
                           indicator=True).query
("_merge == 'left_only'")

Print (df_ex_left.shape)
```

Right Join



Exclusive (query)

how="outer" copy.

Same as left join and left exclusive join.

#Regular Expression (Regex)

#Subtopic : Metacharacters used in regular expression.

- Metacharacters :→ characters with a special meaning.
  eg \d, \w, \s
- We use a website called regex 101

	Examples	Meaning
1	\d	1- helps us get all digit from a text i.e digit(0-9)
2.	\D (Negation of \d)	2- Negation gets the opposite values of each metachar i.e No digits (All the characters / elements that are not digits)
3	\w	3.- Helps us get word character from a text (i.e all word characters that are either (a-z, A-Z, 0-9, _ underscore) Digits and underscore also
4.	\W (Negation of \w)	4. No word character ~(a-z, A-Z, 0-9, _)
5.	\s	5. helps us get whitespaces i.e. (space, tab, new line)
6	\S (Negation)	6. No whitespaces i.e ~(space, tab, new line)
7.	.	7. Matches any character except for a new line
8.	\	8. ignores any special character.
9.	^	9. Matches the beginning of a string. e.g ^Hello string Hello World Hello Result Hello World Hello selected
10	$ Dollar sign	10. Matches the end of a string. e.g Hello$ string Hello World Hello Hello World Hello selected

NB :- ~~Regex flags~~

- Regular expression might change slightly, their behaviour based on some regex flags.
- Regex flags → helps us edit the behaviour of regular expressions

  e.g global regex flag extended regex flag.
  Multiline regex flag Single line regex flag.

- Multiline regex flag → takes a line as a string i.e a line has a beginning and end.
- global regex → takes a paragraph as a string i.e. the start of the paragraph as the start of a string & the end of a paragraph as an end of a string.

NB :- global regex flag is how python behaves by default.

Quantifiers

	Quantifier	Function.
1.	*	= Matches zero or more characters (on the closest character) Greedy.
2.	+	= Matches one or more element (on the closest character) Greedy
3.	?	= Matches zero or one character Lazy.
4	{3}	= Matches the specified number of characters e.g 3 characters. e.g Python!{3} = python!!!
5	{n,}	matches = more than n-characters e.g Python!{3,} result = Python!!!!! → Its greedy.
6	{3,4}	= Range of numbers you wish to match {min number, max number} e.g Python!{2,4} result Python!!!! Regulates the greediness of a quadtifier.

Quantifiers	Functions.
7. ()	= used to capture a specific group of characters eg 987-654-321 . capture the three groups: (\d+)-(\d+)-(\d+) 987=group1 654=group2
8. []	= Matches characters in brackets e.g [851] result 987-654-321 or range e.g [5-9] result = 987-654-321 = Also supports letters range [a-z] [A-Z] or [a-zA-Z] ⇒ All letters

= NB Greedy → Match the longest possible string
~~Lazy~~ → (as many characters as you can)

Lazy → Match the shortest possible string required

eg Hello World.

Greedy ~~(H.+l~~ (H.+l) = Hello Worl
Lazy (H.?l) = Hel or (H.+?l) == Hel

Quantifiers	Functions
	e.g 987-654-321
9. [^]	= Match characters not in bracket e.g [^7-9] result = 987-654-321
10. |	= or condition. e.g Match a string that has 9 or 8 followed by two digits. (9|8)\d{2} result = 987 capture group, matched
11. \b	= Matches word boundary.
12. \B	= Matches no word boundary.
13. \1	= Back Reference ⇒ helps us reuse a capture group we already identified.
13	Sequential Repetitive e.g (123-123-) (123-)\1 = 123-123-

Metacharacters in Python code.

```
import re
```

= When we want to use regular expressions in Python we have to import re module.

```
import re
```

= # Two common methods to match regular expression.

1. Search method (re.search())

```
text = """Hello World.
I love Python!!!!!
My first lucky number is 987-654-321
My second lucky number is 876-543-210
My third lucky number is 765-432-100
My fourth lucky number is 123-456-123-123-456
"""

# return the first match. (first digit of this text)
re.search(r"\d", text) # search(regular exp, dataframe)
# how flags work
re.search(r"^hello", text, flags=re.I)
```

NB `flags=re.I` = Insensitive flag that wont consider the case of the text.

result = "Hello"

2. re.findall() method.

* Returns all the matches.

Return the (first) match of digits (if wont) will returns all digits.

re.findall(r"\d", text)

Exercise.

Match only punctuation.

re.findall(r"[^\w\s]", text) [^\w\s\d]

Match right date format (MM-DD-YYYY) \d{2}

re.findall(r"\d{2}-\d{2}-\d{4}", text)

Match right username format (4-14 characters {letters|digits only}

Prototype (\d|\w){4,14} [a-z|0-9]{4,14}

re.findall(r"[a-zA-Z0-9]{4,14}", text)

Data Cleaning with Pandas

```
import pandas as pd.

df_netflix = pd.read_csv("netflix_titles.csv")

Data cleaning techniques.
1.# Check data types of the columns.
   df_netflix.dtypes
2.# Check the shape
   df_netflix.shape
3 # identifying missing data.
 3.1 # Number of rows missing in each column.(isnull())
   df_netflix.isnull()        #NaN == True.
   df_netflix.isnull().sum() #will return no. of Nan in each column

 3.2# % of rows missing in each column.
   for column in df_netflix.columns:
       x = df_netflix[column].isnull().mean
       Print(x
       Print(column + " : " + str(x))

4.# Dealing with missing data.
4.1 # Drop column with many Nan.  #Not a good approach.
   df_netflix.drop("director", axis=1)
4.2.# Drop specific rows. with NaN.
null_director = df_netflix[df_netflix["director"].isnull()].index
df_netflix.drop(null_director, axis=0) *inplace=True.
4.2.2# Drop specific rows with (~+.isnull())
x =    df_netflix[~(df_netflix["director"].isnull())]
   x.isnull().sum()  #check the columns null values.
```

4.3 # Dropping null values using dropna() simplest
df-netflix.dropna(subset=["director"])

4.4 # Use fillna() to replace NAN by the mean, median (Numeric data) or mode (for categorical string data)
df-netflix["rating"].mode() #we get(TV-MA)
NB fillna() expects what to be filled to be integer/string
#Convert the rating column datatype into a string
ie the mode. (TV-MA) since it was an object.
mode = " ".join(df-netflix["rating"].mode())
type(mode)
df-netflix["rating"].fillna(mode, inplace=True)

NB Be careful not all columns can be filled with the mode.

= We replace ^null values in categorical data with mode and numerical data with mean/median.

4.5 # Replacing null values by an arbitrary number
df-netflix["duration"].fillna("0", inplace=True)

4.6 # Replacing null values backward & forward.
4.6 # Backward.
b-fill
df-netflix["director"].fillna(method="backward")

director	Result	director
Michael Ford.		Michael Ford
Nan		Hans Zimmer
Hans Zimmer		Hans Zimmer.

4.6.2 # Forward.
ffill
df-netflix["director"].fillna(method="forward")

NB Use backward fill and forward fill for data that does not vary much i.e weather in 1mins.

24 Nan 26 → 24 24 26 / 24 26 26.

#5 Extracting data from column with split() & extract()

5.1 # split method.

```
df_netflix["duration"].fillna("0", inplace=True)
df_movie = df["dura
df_movie = df_netflix[df_netflix["type"]=="Movie"]
df_movie
df_movie["minute"] = df_movie["duration"].str.split(expand=True)[0].astype(int)
```

NB split method looks for spaces btwn a sentence
e.g 90_min it splits 90 and min
expand converts the splitted words into two columns

0	1
90	min.

[0] → we chose the 0 column which gives us the duration
and .astype(int) converts that number from string to an integer. & the whole column becomes of datatype int64

Example we have date-added column September 24, 2025 and we want to split it.

df-movie ["date-added"].str.split(",", expand=True)[1]

0	1
September 24	2025

#NB The first split had a blank space thats why we did not add anything inside the split method.

5.2 extract method

NB Extract method are extremely good to use with regular expresion.

#Extract years in the date added column

~~df-ye~~

df-movie ["year_added"] = df-movie ["date-added"].str.extract(r"(\d{4})").astype(int)

=We can also use regex pattern in split method.

6. Identifying outliers.

6.1 Using histogram to identify outliers of numeric data.

```
df_movie["minute"].plot(kind="hist", bins=10)
#get bar ranges with value_counts method and no-of bins
df_movie["minute"].value_counts(bins=10).sort_index()
#Dealing with them(filtering)
df_movie[~((df_movie["minute"]>31) & (df_movie["minute"]<218))]
```

9494453

To find outliers we use negation (~)

6.2 Using boxplots for numeric data